

Experiment 3

Drug Type Classification Using Decision Tree and Random Forest Algorithms

1. Dataset Source

- Source Link: <https://www.kaggle.com/datasets/prathamtripathi/drug-classification>

2. Dataset Description

The Drug Classification dataset contains 200 patient records with 5 features and one target variable. It is a multi-class classification problem where the goal is to predict one of the five drug types (drugA, drugB, drugC, drugX, or drugY) that should be prescribed to a patient. The features include Age (numerical), Sex (categorical), Blood Pressure (BP), Cholesterol level, and Na_to_K ratio (numerical). The dataset is clean, has no missing values, and is ideal for experimenting with classification algorithms like KNN due to its small size and mixed data types.

- **Size:** 200 rows (samples), 6 columns (5 features + 1 target)
- **Target Variable:** Drug (multiclass categorical)
 - Classes: drugA, drugB, drugC, drugX, drugY
 - Represents recommended drug type

Features

- **Age:** Numerical (integer) — patient's age
- **Sex:** Categorical (binary) — M (male), F (female)
- **BP (Blood Pressure):** Categorical (ordinal) — LOW, NORMAL, HIGH
- **Cholesterol:** Categorical — NORMAL, HIGH
- **Na_to_K:** Numerical (float) — Sodium to Potassium ratio

Dataset Characteristics

- Small and clean dataset
- Mixed data types (numerical + categorical)
- No missing values
- Mostly balanced classes (drugY slightly more frequent)
- Suitable for classification problems
- Commonly used for healthcare / personalized medicine applications

3. Mathematical Formulation of the Algorithms

The **Decision Tree** is a supervised learning algorithm that builds a tree-like model for classification. It recursively splits the dataset based on feature values to maximize information gain or minimize impurity.

Key Concepts

Entropy

$$H(S) = - \sum_{i=1}^c p_i \log_2(p_i)$$

Where:

- p_i = proportion of class i in dataset S
- c = number of classes

Information Gain

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- A = feature used for splitting
- S_v = subset of S where feature A has value v

Gini Impurity

$$Gini(S) = 1 - \sum_{i=1}^c p_i^2$$

4. Algorithm Limitations

- Prone to **overfitting**, especially with deep trees (no pruning)
- **Unstable**: Small data changes can create a completely different tree
- Biased toward features with **many categories (high cardinality)**
- Performs poorly on **complex/non-linear datasets** without tuning
- Can be slow for very large datasets (deep trees = high computation)
- Cannot handle **missing values directly** (requires preprocessing)

5. Methodology / Workflow

Step 1: Download the Dataset

Step 2: Set Up Environment

```
!pip install pandas numpy scikit-learn matplotlib seaborn

Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.3)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
```

Step 3: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, ConfusionMatrixDisplay
```

Step 4: Load and Explore the Dataset

```
df = pd.read_csv('drug200.csv')

print("Shape:", df.shape)
print("\nFirst 5 rows:\n", df.head())
print("\nTarget Distribution:\n", df['Drug'].value_counts())
print("\nMissing Values:\n", df.isnull().sum())

Shape: (200, 6)

First 5 rows:
   Age  Sex  BP  Cholesterol  Na_to_K  Drug
0   23   F  HIGH           HIGH    25.355  DrugY
1   47   M  LOW           HIGH    13.093  drugC
2   47   M  LOW           HIGH    10.114  drugC
3   28   F  NORMAL        HIGH     7.798  drugX
4   61   F  LOW           HIGH    18.043  DrugY

Target Distribution:
Drug
DrugY    91
drugX    54
drugA    23
drugC    16
drugB    16
Name: count, dtype: int64

Missing Values:
Age      0
Sex      0
BP       0
Cholesterol  0
Na_to_K  0
Drug     0
dtype: int64
```

Step 5: Separate Features (X) and Target (y)

```
x = df.drop('Drug', axis=1)
y = df['Drug']

# Encode target labels
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

Step 6: Create Preprocessing Pipeline

```
categorical_cols = ['Sex', 'BP', 'Cholesterol']
numerical_cols = ['Age', 'Na_to_K']

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(drop='first'), categorical_cols)
    ])

```

Step 7: Train-Test Split (80% - 20%)

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.2, random_state=42, stratify=y_encoded
)

print("Train samples:", X_train.shape[0])
print("Test samples:", X_test.shape[0])

Train samples: 160
Test samples: 40
```

Step 8: Build and Train Decision Tree Model

```
dt_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('dt', DecisionTreeClassifier(random_state=42))
])

dt_pipeline.fit(X_train, y_train)
dt_pred = dt_pipeline.predict(X_test)
```

Step 9: Build and Train Random Forest Model

```
rf_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('rf', RandomForestClassifier(n_estimators=100, random_state=42))
])

rf_pipeline.fit(X_train, y_train)
rf_pred = rf_pipeline.predict(X_test)
```

Step 10: Evaluate Both Models

```
# Decision Tree Results
print("=== DECISION TREE ===")
print("Accuracy:", accuracy_score(y_test, dt_pred))
print(classification_report(y_test, dt_pred, target_names=le.classes_))

# Random Forest Results
print("\n=== RANDOM FOREST ===")
print("Accuracy:", accuracy_score(y_test, rf_pred))
print(classification_report(y_test, rf_pred, target_names=le.classes_))
```

```
=== DECISION TREE ===
Accuracy: 0.975
      precision    recall  f1-score   support

 DrugY      0.95      1.00      0.97      18
 drugA      1.00      1.00      1.00       5
 drugB      1.00      1.00      1.00       3
 drugC      1.00      1.00      1.00       3
 drugX      1.00      0.91      0.95      11

 accuracy          0.97      40
 macro avg      0.99      0.98      0.99      40
 weighted avg    0.98      0.97      0.97      40

=== RANDOM FOREST ===
Accuracy: 0.975
      precision    recall  f1-score   support

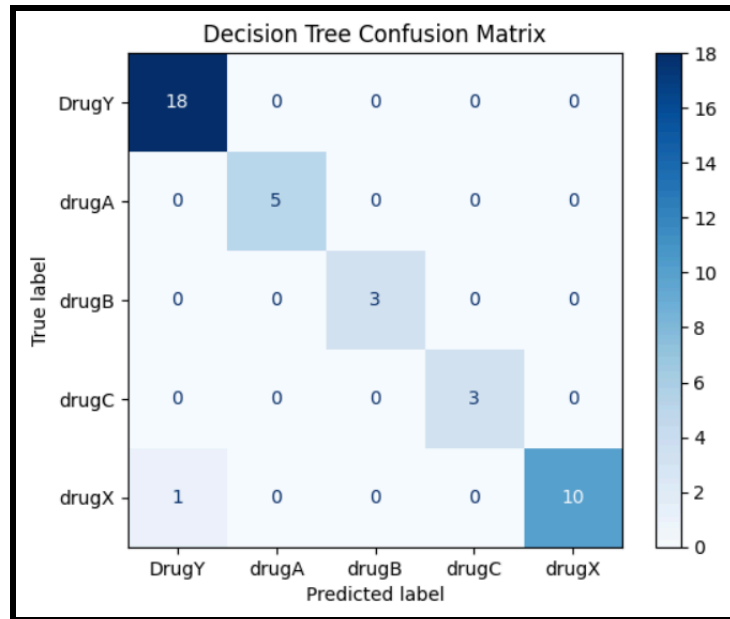
 DrugY      0.95      1.00      0.97      18
 drugA      1.00      1.00      1.00       5
 drugB      1.00      1.00      1.00       3
 drugC      1.00      1.00      1.00       3
 drugX      1.00      0.91      0.95      11

 accuracy          0.97      40
 macro avg      0.99      0.98      0.99      40
 weighted avg    0.98      0.97      0.97      40
```

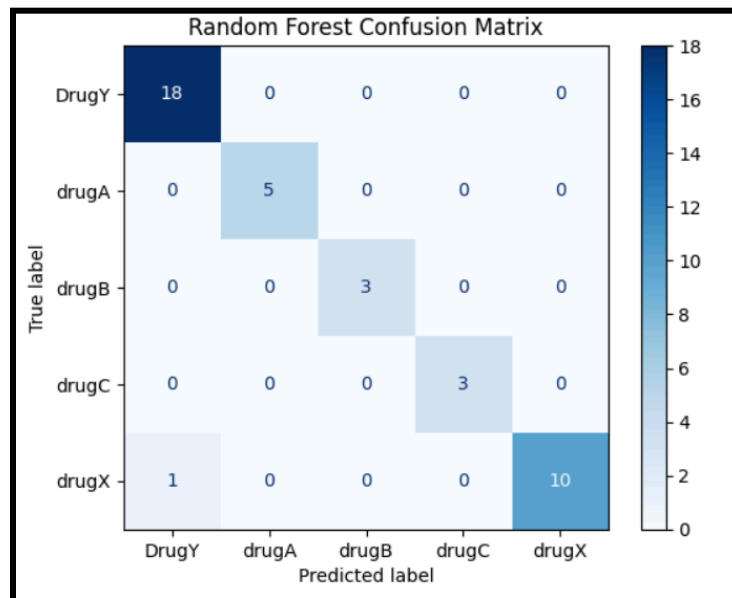
Step 11: Plot Confusion Matrices

```
# Decision Tree Confusion Matrix
cm_dt = confusion_matrix(y_test, dt_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_dt, display_labels=le.classes_)
disp.plot(cmap='Blues')
plt.title('Decision Tree Confusion Matrix')
plt.show()

# Random Forest Confusion Matrix
cm_rf = confusion_matrix(y_test, rf_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm_rf, display_labels=le.classes_)
disp.plot(cmap='Blues')
plt.title('Random Forest Confusion Matrix')
plt.show()
```



The confusion matrix shows the performance of the Decision Tree classifier on the test set. It displays the number of correct predictions (diagonal values) and misclassifications (off-diagonal values) for each of the five drug types. The model correctly classified most instances, with only one misclassification (drugX predicted as drugY).



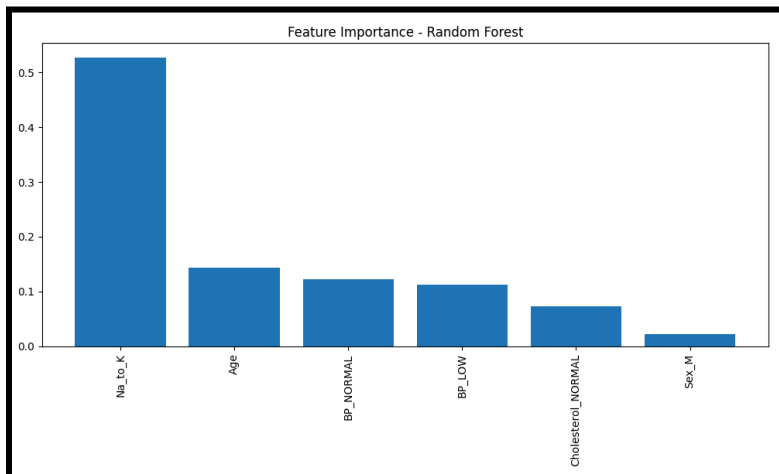
The confusion matrix illustrates the performance of the Random Forest classifier on the test set. The model achieved perfect classification for most drug categories, with only one misclassification (drugX predicted as drugY). Overall, it demonstrates strong and stable predictive capability.

Step 12: Feature Importance (Random Forest)

```
rf_model = rf_pipeline.named_steps['rf']
feature_names = numerical_cols + list(
    rf_pipeline.named_steps['preprocessor']
    .named_transformers_['cat'].get_feature_names_out(categorical_cols)
)

importances = rf_model.feature_importances_
indices = np.argsort(importances)[::-1]

plt.figure(figsize=(10,6))
plt.title("Feature Importance - Random Forest")
plt.bar(range(len(importances)), importances[indices])
plt.xticks(range(len(importances)), [feature_names[i] for i in indices], rotation=90)
plt.tight_layout()
plt.show()
```



This bar chart displays the relative importance of each feature as determined by the Random Forest model. The Na_to_K ratio is by far the most important feature, contributing more than 50% to the model's decisions. Age, Blood Pressure (BP), and Cholesterol follow as moderately important features, while Sex has the least impact.

6. Performance Analysis

```
best_dt = grid_dt.best_estimator_
best_rf = grid_rf.best_estimator_

print("Final DT Test Accuracy:", accuracy_score(y_test, best_dt.predict(X_test)))
print("Final RF Test Accuracy:", accuracy_score(y_test, best_rf.predict(X_test)))

Final DT Test Accuracy: 0.975
Final RF Test Accuracy: 0.975
```

Both **Decision Tree** and **Random Forest** achieved **97.5% accuracy**, showing excellent classification performance.

- Most classes (**DrugY, DrugA, DrugB, DrugC**) were classified perfectly (Precision, Recall, F1 = 1.00).
- **DrugX** had slightly lower recall (0.91), indicating minor misclassification.
- Confusion matrix confirms very few errors overall.

Both models perform well, with Random Forest offering better generalization and Decision Tree being more interpretable.

7. Hyperparameter Tuning

```
# For Decision Tree
param_grid_dt = {
    'dt__max_depth': [3, 5, 7, 10, None],
    'dt__min_samples_split': [2, 5, 10],
    'dt__criterion': ['gini', 'entropy']
}

grid_dt = GridSearchCV(dt_pipeline, param_grid_dt, cv=5, n_jobs=-1)
grid_dt.fit(X_train, y_train)
print("Best DT Params:", grid_dt.best_params_)

# For Random Forest
param_grid_rf = {
    'rf__n_estimators': [50, 100, 200],
    'rf__max_depth': [5, 10, None],
    'rf__min_samples_split': [2, 5]
}

grid_rf = GridSearchCV(rf_pipeline, param_grid_rf, cv=5, n_jobs=-1)
grid_rf.fit(X_train, y_train)
print("Best RF Params:", grid_rf.best_params_)

Best DT Params: {'dt__criterion': 'gini', 'dt__max_depth': 5, 'dt__min_samples_split': 2}
Best RF Params: {'rf__max_depth': 5, 'rf__min_samples_split': 2, 'rf__n_estimators': 50}
```

Using **GridSearchCV**, the best parameters were:

- **Decision Tree:**
criterion = gini, max_depth = 5, min_samples_split = 2
- **Random Forest:**
max_depth = 5, min_samples_split = 2, n_estimators = 50

Tuning improved model stability and controlled overfitting while maintaining high accuracy.